06 · 工作流编排与多 Agent 协作

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：谁来决定「做什么、谁来做、按什么顺序做」，以及怎么保证这个决定可解释、可恢复。

1. 什么时候用固定 Workflow，什么时候用 Agent?

我会按「任务的可枚举性」和「失败的代价」来判断。

固定 Workflow 适合：步骤稳定、依赖明确、需要可复现和审计的场景，比如测试执行、发布流程、审批流。它的优点是确定性高、容易测试和回滚，缺点是对变化不灵活。

Agent 适合：目标明确但路径不固定、需要根据中间结果决定下一步的场景，比如故障诊断、开放式检索、复杂问题的多假设排查。它的优点是适应性强，缺点是结果不稳定、成本高、难复现。

实际系统通常是混合：主干用 Workflow，局部节点用 Agent。比如测试平台的主流程是「需求 → 设计 → 用例 → 执行 → 分析」，这是固定的；但「失败后先查什么」交给 Agent 决策。

关键是不能让 Agent 在关键路径上「偷偷降级成自由发挥」，也不能为了灵活性把本来确定的流程改成不确定的。

2. 设计一个 DAG 编排系统，核心是什么?

核心是四件事：节点定义、依赖关系、数据流、校验。

节点定义要说清楚：这个节点需要什么输入、产出什么输出、调用哪个 Agent 或工具、成功标准是什么、失败怎么处理。

依赖关系决定执行顺序和并行度。DAG 要能检测环、检测孤立节点、检测依赖是否闭合。

数据流是最容易出问题的地方：上游的输出必须能满足下游的输入 Schema。只检查「有没有连边」不够，还要检查字段类型、必填项和语义是否匹配。

校验至少包括：无环、依赖闭合、输入输出匹配、权限满足、资源预算合理、关键节点有兜底。

执行层还要有状态持久化、超时、重试、并行控制和取消。DAG 不是画出来好看，而是要在失败时知道该重跑哪一段。

3. 动态 Workflow 怎么生成？如何防止模型乱规划?

我会采用「LLM 提议 + 确定性校验 + 有限修复」的模式。

先给模型一个能力注册表：每个 Agent/工具的能力标签、输入输出 Schema、前置条件、成本、权限。模型只能从注册表里选，不能凭空发明节点。

模型生成计划后，校验器检查：依赖是否闭合、输入输出是否匹配、是否有环、权限是否满足、预算是否超限。校验失败时，把具体错误返回给模型做一次修复，而不是整份重规划。

如果多次修复仍失败，就退回到预定义模板或人工编排，不能让非法计划进入执行。

为了让结果可解释，我会记录每次选择的原因：为什么选这个 Agent、为什么用这个顺序、哪些候选被排除了。

动态生成适合「没有合适模板」的场景。生产环境优先用预定义或模板化 Workflow，动态生成是兜底，不是默认。

4. Agent 路由怎么做？为什么不能只靠 LLM 选？

路由要回答「谁来干」，我会分三步：候选过滤、评分排序、置信度决策。

候选过滤先用硬条件：能力标签、输入输出 Schema、权限、健康状态、租户限制。不满足的直接排除，不让模型在非法候选里选。

评分可以用多个分量：能力匹配度、历史成功率、当前负载、成本、时延、模板复用奖励。每个分量有权重，最后得到一个可解释的分数。

LLM 可以用来做语义匹配和意图理解，但不应该单独决定路由。原因是它不稳定、不可审计，而且容易被工具名字误导。更稳的做法是「规则优先 + LLM 补充」：明确意图走规则，模糊意图再用模型，并且保留置信度。

置信度低时不要硬路由，应该澄清或给用户选项。路由错了，后面所有步骤都白做。

5. 多 Agent 之间怎么通信？共享上下文还是隔离？

常见有三种方式：共享状态、消息传递、Handoff。

共享状态适合同一任务内的协作，所有 Agent 读写同一份任务状态，简单直接，但容易互相污染，权限也不好隔离。

消息传递适合解耦，Agent 之间通过结构化消息通信，上下文隔离好，但需要定义消息协议和状态同步。

Handoff 是控制权转移，当前 Agent 把任务和必要上下文交给下一个 Agent，适合分阶段处理。关键是交接内容要明确：目标、已完成、待办、约束、证据。

我倾向于「默认隔离，按需共享」：每个子 Agent 只拿到完成自己任务所需的最小上下文，结果通过结构化输出汇总到编排层。这样既能控制 Token，也能减少错误传播。

无论哪种方式，都要有统一的追踪 ID，能把一次任务的跨 Agent 轨迹串起来。

6. 多 Agent 如何防止死循环、死锁和无限委派？

我会加五道限制。

第一，深度限制：委派链最多几层，超过就拒绝或转人工。

第二，预算限制：整个任务有最大步数、时间、Token 和成本，子 Agent 共享总预算，不能各自无限花。

第三，访问集限制：记录已经调用过的 Agent 和工具组合，重复出现就中断或换策略。

第四，状态检测：如果连续几步状态没有实质变化，判定为停滞，触发重规划或退出。

第五，显式取消：用户或上层可以随时取消，取消信号要传到所有子任务和上游模型请求。

死锁常见于多个 Agent 互相等待对方的结果。解决办法是明确依赖方向、避免循环依赖，给等待设置超时，超时后降级。

这些限制要在编排层统一做，不能指望每个 Agent 自己遵守。

7. 人在回路（Human-in-the-loop）怎么设计?

先确定哪些节点需要人：高风险动作、低置信度决策、合规要求、不可逆操作。

交互形式有三种：审批（同意/拒绝）、补充信息（澄清）、修正（改参数或改计划）。

工程上要支持「暂停–持久化–恢复」：任务到人工节点时保存完整状态，人可以几小时后再处理，恢复时从断点继续，而不是重跑。

审批要展示足够的信息：Agent 想做什么、为什么、影响范围、参数、风险、回滚方案。不能只给一个「是否通过」。

超时要有默认策略：低风险可以自动通过，高风险必须拒绝或升级，不能无限等待。

所有人工操作要记录：谁、什么时候、改了什么、理由是什么。这既是审计要求，也是后续优化的数据。

8. 如何保证编排过程可复现、可审计?

三个关键：版本、快照、轨迹。

版本包括 Prompt 版本、模型版本、Agent 配置版本、工具版本、Workflow 版本。任何一次执行都要能回答「当时用的是哪一版」。

快照是执行时的完整输入和状态：用户输入、检索到的文档、上下文、计划、工具参数。只记录最终输出是不够的，复现需要中间状态。

轨迹是每个节点的开始/结束、输入输出、耗时、错误、重试、路由决策。用统一的 trace ID 串起来，能还原整条链路。

有了这三样，才能做归因：失败是理解错了、检索错了、路由错了，还是工具错了。

可复现不等于完全确定性，模型仍有随机性；但至少能知道「同样的输入和版本下，系统做了什么」。

9. 多 Agent 执行失败怎么处理？重试、补偿还是回滚?

先分类，再决定。

可重试的失败：瞬时网络错误、限流、超时。用退避重试，但要幂等。

不可重试的失败：参数错误、权限不足、业务规则不满足。重试没用，要把原因返回给编排层，触发重规划或转人工。

部分成功的失败：一些节点成功、一些失败。这时要考虑补偿：撤销已完成但有副作用的操作，或者保留已完成结果、只重跑失败部分。

回滚不是总能做到，尤其是外部副作用。所以关键操作要设计成可补偿（比如取消订单、删除临时资源），不能假设一定能回滚。

编排层还要有整体超时和熔断，避免一个子任务卡住拖垮整个任务。

我的原则是：能幂等重试的就重试，不能的就补偿，补偿不了的就必须人工介入。

10. 怎么评估编排系统的效果?

我会看四类指标。

效果：任务成功率、端到端完成率、人工介入率、人工修改率。

效率：平均步骤数、并行度、端到端时延、单任务成本。

稳定性：失败率、重试率、超时率、恢复成功率、死循环拦截次数。

可解释性：路由决策可解释比例、失败归因覆盖率、轨迹完整率。

除了数字，还要做案例分析：挑失败任务看轨迹，判断是任务定义、Agent 能力、路由还是工具的问题。

评估集要覆盖不同类型和难度的任务，不能只测简单场景。编排系统的价值往往体现在复杂任务和异常路径上。

11. 设计一个测试全流程多 Agent 平台，你会怎么拆?

我会按测试阶段拆 Agent，但不会一个阶段一个 Agent 拆得很碎。

核心 Agent 包括：需求分析 Agent（提取测试点）、用例设计 Agent（生成步骤和预期）、环境准备 Agent（检查/申请环境）、执行 Agent（调用测试工具）、结果分析 Agent（判断失败原因）、缺陷整理 Agent（生成缺陷单）。

它们之间通过编排层连接，不直接互相调用。编排层根据输入资产决定需要哪些阶段：如果用户已经给了用例，就跳过需求分析和用例设计，只跑环境、执行、分析。

共享能力下沉：模型网关、工具执行、知识检索、记忆、评测、可观测性都是公共层，不放在某个 Agent 里。

可靠性上，每个阶段有输入输出契约，失败可重试或跳过；长任务有检查点；高风险操作（改环境、提缺陷）需要审批。

这个拆分的关键是「按能力边界拆，不按角色数量拆」。Agent 太多，协调成本会超过收益。

12. 路由置信度低的时候怎么办?

不要硬路由，先澄清。

我会设一个置信度阈值：高于阈值直接路由；中间区间给用户几个候选，让他确认；低于阈值就追问，补充关键信息。

澄清问题要具体，比如「你是想生成新用例，还是执行已有用例？」，而不是「你能再说清楚点吗」。

如果用户不方便确认，可以走默认策略：选最安全的路径，比如只读、只生成草稿、不执行副作用操作，并明确告知。

还要记录「澄清后是否正确」的数据，用来优化意图识别和路由阈值。

低置信度硬路由的代价很高：一旦走错，后面所有步骤都浪费，还可能产生副作用。宁可多问一句。

高频追问

- 固定 Workflow 里能不能嵌入 Agent 节点？状态怎么传递?
- Agent Pool 的能力标签应该由谁维护？怎么防止标签失真?
- 多 Agent 共享记忆还是各自独立记忆?
- Handoff 时应该传完整上下文还是摘要?
- 如何判断一个任务「适合动态规划」而不是「应该写死」？
- 编排层如何做超时？子任务超时后上游怎么办?